NAME

dedupe − delete files from a cull tree that duplicate an actual tree

SYNOPSIS

dedupe −−database *DB* [**−−delete**] [**−−allow−mixed**] [**−n**, **−−offline**] *ACTUAL CULL...*

DESCRIPTION

dedupe dismantles one or more redundant copies of a directory tree. *ACTUAL* is the copy being kept — it is never modified, ever. Each *CULL* is a redundant copy: every file in it whose digest duplicates a file in the *corresponding* directory of *ACTUAL* is deleted, regardless of name, and directories emptied by this are removed on the way back up. If a cull holds three files matching one actual-side witness, all three are deleted. What survives in cull is therefore always a signpost — something unique, unknown, or suspicious — and the cull root itself always survives, even empty, as a placeholder and a receipt.

More than one *CULL* may be given: each is walked in sync against the same *ACTUAL* in turn, and the run's counters and summary aggregate across them.

dedupe deletes files. It is an experimental companion to **sumtag**(1) and works from a sumtag SQLite database that already knows both trees; it never computes a checksum. A bare run is a full preview (**would delete**, **would sweep**, **would rmdir**, with the database opened read-only); actual deletion requires the explicit **−−delete** option.

THE SYNCHRONIZED WALK

Both trees are walked depth-first at the same time, locked in step by relative path: while visiting *ACTUAL***/foo/bar** the walk is in *CULL***/foo/bar**, and it never descends into a subdirectory name that does not exist as a real directory on both sides. Comparison and deletion are therefore flat, one directory pair at a time: a cull file dies exactly when a same-digest file sits in the corresponding actual directory — same name not required, same relative directory required. Reorganised duplicates are out of scope by design, in exchange for a comparison that can be verified by looking at one directory pair at a time.

The one exception is the empty-directory carve-out: a cull-side subdirectory with no real files anywhere beneath it (only ignorable junk files such as **.DS_Store**, qualifying symbolic links, and empty directories) is swept even without an actual-side counterpart — deleting it destroys nothing.

WHAT IS DELETED, AND WHAT NEVER IS

Only plain files positively identified as duplicates are deleted, plus the exit junk below, and only ever on the cull side.

- Files the database does not know (never stamped or mirrored) are invisible: never matched, never deleted, left in place even if that leaves their directory standing.
- Ignorable junk files (**.DS_Store**) never block a directory's removal, and are swept exactly when they are the last thing standing before the directory is removed — never earlier.
- Symbolic links are never followed and never digest-matched. A link is swept at directory exit only if it is relative and points inside the cull tree — where it points is what matters, not whether the target still exists, since the deletion phase has usually just emptied it. An absolute link is never deleted, wherever it points, and neither is any link escaping the cull tree: such a link blocks its directory like any other survivor.
- A cull directory containing an **@sumtag−ignore** marker (see **sumtag**(1)) is fenced off: not descended, nothing inside it deleted, swept, or removed; it blocks its parent like any survivor. A marker on the cull root itself is honoured with a warning, and the run does nothing.

TRUST MODEL

Stored digests are trusted; neither side is re-hashed. That is defensible because deleting a true duplicate is reversible — the actual tree keeps the bytes — but it only holds if the stored digests describe the current bytes, so three vetoes gate every candidate, none of which read file contents:

- the database digest must equal the digest in the file's own extended attribute, on both sides;

- the file's live mtime must equal the mtime recorded in its extended attribute, on both sides;
- the cull file's live size must equal its witness's. XXH3-64 is a 64-bit non-cryptographic hash, and the size check is free collision insurance: a digest match with differing sizes is reported loudly as a mismatch, the file is kept, and the run exits 2.

A cull-side file failing a veto is kept, with a warning; an actual-side file failing a veto simply witnesses nothing.

SAFETY CHECKS

All of the following are verified before anything is at risk:

- Every root must exist, and *ACTUAL* must be disjoint from each cull under **realpath**(3) — not equal, and neither nested inside the other, so symbolic links cannot disguise an overlap. Culls are not checked against one another: two overlapping culls only cost redundant work, never the copy being kept.
- The database must already exist and must know every root: each root's live mount point must be recorded and file rows must be present beneath it. An unmounted drive is an error, never a silent no-match.
- If the roots' rows carry more than one digest algorithm across *ACTUAL* and all the culls, the run is refused unless **−−allow−mixed** is given, because cross-algorithm duplicates would silently survive.
- At the moment of deletion, if the cull file and its witness resolve to the same path under **realpath**(3), they are one directory entry reached through links; the deletion is refused loudly (exit status 2), since deleting "the copy" would delete the only name. The same inode behind different real paths is a genuine hard link — safe to delete, and noted.

Pass the roots spelled as sumtag scanned them: row lookups use the absolute path as given, exactly as sumtag records paths, so a differently-spelled root (for example through a symlinked path component) fails the must-know-both-roots check with a clear error rather than matching nothing.

OFFLINE PREDICTION

With **−n** (**−−offline**), **dedupe** answers "what *might* be deleted" from the database alone: no filesystem access at all, so neither root needs to be mounted. The roots are resolved against the *stored* mount points, the flat same-relative-directory match runs over the database rows, and each candidate prints as **might delete** — deliberately not **would delete**, because this is a prediction, not a plan. Everything the filesystem would have contributed is skipped: the three trust-model vetoes, the **realpath**(3) identity checks, **@sumtag−ignore** fences, and all sweep and directory-removal predictions (files the database never stamped are invisible, so it can never know a directory would empty). Two row-only checks remain: the collision-insurance size comparison, where a **−−locate** run has recorded sizes on both sides, and a note when a candidate shares its recorded inode with its witness (a hard link or an alias — a live run decides which). The bare live preview remains the only exact one.

The run is announced with an **offline:** line so a consumer of the prediction knows what it is consuming, the database is opened read-only, and **−n** conflicts with **−−delete** — a prediction cannot arm. Exit status is unchanged: 1 means candidates were found, 2 covers errors and refusals (including a recorded-size mismatch).

OPTIONS

−−database *DB*
: Path to the sumtag SQLite database that knows every tree. Required.

−−delete
: Actually delete. Without it, the run is a complete preview and the database is opened read-only. Deleted files' database rows are removed in the same run, committed per directory, so an interrupted run stays consistent.

−−allow−mixed
: Proceed even when the roots carry more than one digest algorithm. Cross-algorithm duplicates silently survive; see **SAFETY CHECKS** above.

−n, **−−offline**
: Predict what might be deleted from the database alone, with no filesystem access; neither root needs to be mounted. See **OFFLINE PREDICTION** above. Conflicts with **−−delete**.

RUN SUMMARY

Every run ends with a summary block: files deleted (or would-delete) and their cumulative size, items swept, directories removed, files kept (broken down as unique, unknown, or stale), directories fenced by **@sumtag−ignore** markers, errors, and the database and roots in use. Interrupting a run with Ctrl-C prints the same summary, prefixed by an **interrupted** line; the counters cover only completed work, and commits are per directory, so the database matches what actually happened.

Immediately above the summary, the run echoes the exact command line it was invoked with under a **command line:** heading, shell-quoted so it can be copied and re-run as-is — a path containing whitespace stays a single argument. This is handy for turning a preview into the armed run: copy the line and add **−−delete**.

As the run works, each action line — the **delete**, **sweep**, or **rmdir** it prints for a path — ends that path with an **ls**(1) **−F** style type indicator: ***** for an executable file, **@** for a symbolic link, **/** for a directory, **|** for a FIFO, and **=** for a socket; a plain file gets none. Offline prediction (**−n**) omits the indicator, since reading a path's type would mean touching the filesystem it deliberately never opens.

EXIT STATUS

0 nothing redundant was found.

1 duplicates were found (deleted, or would-delete under preview).

2 errors or a safety refusal prevented a complete, clean run.

130 the run was interrupted by Ctrl-C (128 + SIGINT, the shell convention).

EXAMPLES

Preview what dismantling a redundant backup copy would delete:

```
dedupe --database /var/lib/sumtag.sqlite /data/proj /backup/proj-old
```

Actually delete:

```
dedupe --database /var/lib/sumtag.sqlite --delete /data/proj /backup/proj-old
```

Predict what might be deleted while the backup drive is unplugged:

```
dedupe --database /var/lib/sumtag.sqlite -n /data/proj /backup/proj-old
```

Dismantle several redundant copies against one kept tree in a single run:

```
dedupe --database /var/lib/sumtag.sqlite --delete /data/proj \
    /backup/proj-old /mnt/archive/proj-2024
```

NOTES

dedupe is experimental — and it is the dangerous companion: always preview first. Both trees should be freshly stamped (**sumtag −−sum −−database** ...) before a run, so the trust-model vetoes have current metadata to check against; stale stamps cause files to be kept, never wrongly deleted.

SEE ALSO

sumtag(1), **grouper**(1)

AUTHOR

The sumtag authors.